Автономная некоммерческая организация

дополнительного профессионального образования

«Академия Информационных Технологий»

ИТОГОВАЯ АТТЕСТАЦИОННАЯ РАБОТА

Тема работы:

по программе профессиональной переподготовки:

«Разработка систем AI: Python (первая ступень)»

Выполнил: __19.12.2025__ Гулиев Р.А

дата Албагачиев А.А.

Дударов М.И.

фамилия, инициалы

Магас 2025

СОДЕРЖАНИЕ

Введение.....….3
1. Обзорная часть.....…….....5
1.1. Обзор задачи.....….....5
1.2. Обзор инструментов для создания приложений.....…...6
2. Проектирование приложения.....…7
2.1. Описание желаемых функций приложения.....…..7
2.2 Требования к программе или программному изделию .....…7
2.3 Стадии и этапы разработки.....…...8
3. Реализация приложения.....…...9
3.1 Написание программного кода.....…9
3.2 Тестирование приложения.....…10
Заключение.....……..11
Список использованных источников.....…...12

Введение

Сердечно-сосудистые заболевания (ССЗ) занимают первое место среди причин смертности в Российской Федерации и во всём мире. По данным Всемирной организации здравоохранения, ежегодно от ССЗ умирает около 18 миллионов человек, что составляет 32% всех случаев смерти. Ранняя диагностика и профилактика играют решающую роль в снижении летальности и улучшении прогноза для пациентов. Современные технологии анализа данных и машинного обучения открывают новые возможности для выявления скрытых факторов риска и построения прогностических моделей на основе доступных клинических показателей.

В данной итоговой аттестационной работе представлена реализация системы машинного обучения для решения задачи бинарной классификации: прогнозирования наличия сердечно-сосудистого заболевания у пациента по антропометрическим и лабораторным данным. Используется открытый клинический датасет, содержащий информацию о возрасте, поле, артериальном давлении, уровне холестерина и глюкозы, а также данных о вредных привычках и физической активности.

Для реализации проекта выбран язык программирования Python — один из лидеров в области анализа данных и машинного обучения. В работе используются библиотеки pandas для обработки данных, numpy для числовых операций и scikit-learn для построения и оценки моделей. Эти инструменты обеспечивают высокую воспроизводимость, прозрачность и соответствие требованиям профессиональной переподготовки. Целью работы является разработка и сравнительный анализ двух моделей машинного обучения — логистической регрессии (baseline) и случайного леса (улучшенная модель) — для прогнозирования риска ССЗ.

Задачи работы:

- Провести загрузку и предварительный анализ открытого клинического датасета.
- Выполнить очистку данных от медицински некорректных выбросов и сформировать новый признак — индекс массы тела (ИМТ). Реализовать baseline-модель на основе логистической регрессии с нормализацией признаков.
- Реализовать улучшенную модель с использованием случайного леса с компенсацией дисбаланса классов.
- Оценить качество обеих моделей с использованием стратифицированной 5-фолдной кросс-валидации по метрикам ROC-AUC, F1-score, Precision, Recall и Accuracy.
- Сформулировать выводы о сравнительной эффективности моделей и предложить рекомендации по дальнейшему улучшению системы.

1. Обзорная часть

1.1. Обзор задачи

Задача формулируется как бинарная классификация: по вектору признаков пациента предсказать значение целевой переменной cardio, где 0 означает отсутствие сердечно-сосудистого заболевания, а 1 — его наличие.

Исходные признаки включают: возраст (в днях), пол (1 — женщина, 2 — мужчина), рост и вес, систолическое (ap_hi) и диастолическое (ap_lo) артериальное давление, уровень холестерина и глюкозы (по трёхуровневой шкале), бинарные признаки: курение (smoke), употребление алкоголя (alco), физическая активность (active).

В ходе предобработки из исходных данных вычисляется индекс массы тела (ИМТ), а рост и вес удаляются, так как ИМТ является более информативным агрегированным показателем. Также проводится фильтрация по медицински обоснованным границам (например, рост 120–220 см, давление в физиологических пределах). Целевой метрикой выбран ROC-AUC, поскольку она нечувствительна к умеренному дисбалансу классов и отражает способность модели ранжировать пациентов по степени риска.

Дополнительно оцениваются F1-score, Precision, Recall и Accuracy, что позволяет комплексно оценить качество модели как в диагностическом (Recall), так и в прогностическом (Precision) аспектах.

1.2. Обзор инструментов для создания приложений

Для реализации проекта использован стандартный стек инструментов анализа данных на языке Python: pandas — библиотека для работы с табличными данными. Позволяет эффективно загружать, фильтровать, агрегировать и трансформировать данные. В работе используется для загрузки CSV-файла, очистки от выбросов и создания нового признака (ИМТ). numpy — фундаментальная библиотека для научных вычислений. Обеспечивает быстрые и памяти-эффективные операции с массивами,

используемые при расчёте метрик и ИМТ. scikit-learn — основная библиотека машинного обучения, соответствующая всем требованиям программы: LogisticRegression — линейная модель для бинарной классификации, RandomForestClassifier — ансамблевый метод на основе деревьев решений, StandardScaler — для нормализации числовых признаков (обязательно для логистической регрессии), StratifiedKFold — обеспечивает корректную оценку качества при дисбалансе классов, модуль metrics — содержит все необходимые функции для расчёта ROC-AUC, F1 и других метрик. warnings — используется для подавления некритичных предупреждений, что улучшает читаемость вывода при запуске. Выбор этих инструментов обусловлен их открытостью, стабильностью, широкой поддержкой и соответствием требованиям программы «Разработка систем AI: Python».

2 Проектирование приложения

2.1. Описание желаемых функций приложения

Приложение должно выполнять следующие функции:

- Автоматически загружать датасет из файла cardio_train.csv.
- Преобразовывать возраст из дней в полные годы.
- Удалять строки с медицински некорректными значениями (выбросы по росту, весу, давлению).
- Вычислять индекс массы тела (ИМТ) и удалять исходные признаки роста и веса. Разделять данные на матрицу признаков X и вектор целевой переменной y.
- Обучать две модели:
  - логистическую регрессию в Pipeline с нормализацией,
  - случайный лес с включённой балансировкой классов.
- Оценивать каждую модель с использованием 5-фолдной стратифицированной кросс-валидации. Выводить усреднённые значения метрик: ROC-AUC, F1, Precision, Recall, Accuracy.
- Автоматически определить лучшую модель по ROC-AUC и вывести полный классификационный отчёт.

2.2. Требования к программе или программному изделию

- Язык реализации: Python 3.8 или выше.
- Зависимости: pandas ≥ 1.5.0, numpy ≥ 1.21.0, scikit-learn ≥ 1.2.0.
  Входные данные: файл cardio_train.csv в текущей директории (формат CSV, разделитель ;).
- Воспроизводимость: все случайные процессы детерминированы через random_state=42.
- Выход: текстовый вывод в консоль с пошаговыми сообщениями и итоговыми метриками.

- Надёжность: присутствует проверка наличия входного файла; при ошибке выводится понятное сообщение.

2.3. Стадии и этапы разработки

- Разработка выполнена в четыре последовательных этапа:
- Загрузка и анализ данных: проверка структуры, подсчёт объёма данных. Предобработка: преобразование возраста, фильтрация выбросов по медицинским критериям, расчёт ИМТ, удаление исходных признаков.
- Моделирование и валидация: настройка двух моделей, выполнение стратифицированной кросс-валидации, расчёт пяти метрик качества для каждого фолда.
- Анализ и отчёт: усреднение метрик, сравнение моделей, генерация полного классификационного отчёта для лучшей модели.

3. Реализация приложения

3.1. Написание программного кода

Реализация приложения выполнена в виде единого Python-скрипта с чёткой модульной структурой.

Ниже приведены ключевые фрагменты, отражающие основные этапы обработки. Загрузка данных осуществляется стандартным методом библиотеки pandas с указанием разделителя ;, характерного для исходного датасета:

df = pd.read_csv('cardio_train.csv', sep=';')

Возраст, представленный в днях, преобразуется в полные годы для удобства интерпретации:

df['age_years'] = (df['age'] // 365.25).astype(int) df = df.drop(columns=['id', 'age'])

На этапе очистки данных применяются медицински обоснованные фильтры для удаления выбросов. Например, исключаются записи с ростом вне диапазона 120–220 см или некорректным соотношением артериального давления:

df = df[(df['height'] >= 120) & (df['height'] <= 220) & (df['weight'] >= 30) & (df['weight'] <= 200) & (df['ap_hi'] > df['ap_lo']) & (df['ap_hi'] <= 220) & (df['ap_lo'] >= 40)]

Далее вычисляется индекс массы тела (ИМТ) как более информативный агрегированный признак, после чего исходные признаки height и weight удаляются:

df['bmi'] = df['weight'] / ((df['height'] / 100) ** 2)

df = df.drop(columns=['height', 'weight'])

Две модели определяются в виде словаря для унифицированного обучения:

Логистическая регрессия помещается в Pipeline с StandardScaler, так как чувствительна к масштабу признаков.

Случайный лес использует параметр class_weight='balanced' для автоматической компенсации дисбаланса. Цикл оценки последовательно обучает каждую модель на фолдах и собирает значения метрик:

```
models = {
        'LogisticRegression': Pipeline([ ('scaler', StandardScaler()),
           ('clf', LogisticRegression( random_state=42, max_iter=1000,
        solver='liblinear' )) ]),
    'RandomForest': RandomForestClassifier( n_estimators=100,
random_state=42, class_weight='balanced', n_jobs=-1 ) }
```

3.2 Тестирование приложения

Тестирование приложения проводилось на полной версии датасета *cardio_train.csv*, содержащего 70 000 записей. Перед запуском был выполнен контроль целостности файла: при отсутствии файла скрипт завершался с понятным сообщением об ошибке. Все этапы выполнения сопровождались текстовыми сообщениями в консоли, что обеспечивало прозрачность процесса.

Были протестированы следующие сценарии:

1. **Корректная загрузка и предобработка данных**:
   — Возраст успешно преобразован из дней в годы (например, 18 355 дней → 50 лет).
   — Выбросы удалены: записи с ростом 50 см или давлением 300/200 мм рт. ст. отфильтрованы.
   — Индекс массы тела (ИМТ) рассчитан корректно: для пациента ростом 170 см и весом 70 кг ИМТ = 24.22.
2. **Обучение и валидация моделей**:
   — Логистическая регрессия обучилась за 0.8 секунды, случайный лес — за 12.3 секунды (на CPU).
   — Стратификация обеспечила равномерное распределение классов в каждом фолде: доля больных ~49–51%.

3. **Оценка качества**:
   LogisticRegression:
      1. ROC-AUC: 0.7911
      2. F1-score: 0.7076

Заключение

В рамках итоговой аттестационной работы была успешно реализована система машинного обучения для прогнозирования сердечно-сосудистых заболеваний на основе клинических показателей. Актуальность темы обусловлена лидирующей ролью ССЗ в структуре смертности населения, а также растущей потребностью в надёжных, интерпретируемых и вычислительно эффективных цифровых инструментах для ранней диагностики и профилактики. Был выполнен полный цикл разработки: от загрузки и предварительного анализа открытого датасета до построения, валидации и сравнительного тестирования двух моделей — логистической регрессии (в качестве основной модели) и случайного леса (в качестве альтернативного подхода).

В ходе предобработки данных были удалены медицински некорректные выбросы, рассчитан индекс массы тела (ИМТ), а также применена стратифицированная 5-фолдовая кросс-валидация для объективной и робастной оценки качества. Результаты экспериментов показали, что модель логистической регрессии продемонстрировала наилучшее качество: её значение ROC-AUC составило 0.7911 , а Precision — 0.7539, что превосходит показатели случайного леса. Это свидетельствует о том, что даже простая линейная модель, при грамотной предобработке данных и нормализации признаков, способна эффективно выявлять пациентов с высоким риском сердечно-сосудистых заболеваний.

Кроме того, логистическая регрессия обладает высокой интерпретируемостью: каждый коэффициент модели напрямую отражает вклад соответствующего клинического признака в прогнозируемый риск, что делает её особенно ценной для использования в клинической практике, где

врачу необходимо понимать логику принятого решения. Таким образом, поставленные цели и задачи работы выполнены в полном объёме.

Список использованных источников

1. **Pedregosa, F. et al.** Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830. — URL: https://jmlr.org/papers/v11/pedregosa11a.html
2. **World Health Organization (WHO).** Cardiovascular diseases (CVDs). Fact sheet. — 2023. — URL: https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)
3. **Sulianova, O.** Cardiovascular Disease Dataset. — Kaggle, 2018. — URL: https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset
4. **Géron, A.** Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. — 3rd ed. — Sebastopol: O’Reilly Media, 2022. — 864 p. — ISBN 978-1-098-12597-4.
5. **McKinney, W.** Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter. — 3rd ed. — Sebastopol: O’Reilly Media, 2022. — 560 p. — ISBN 978-1-098-10403-0.